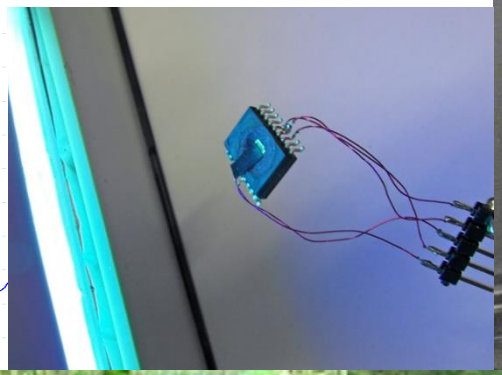
Fault injection 1

David Oswald

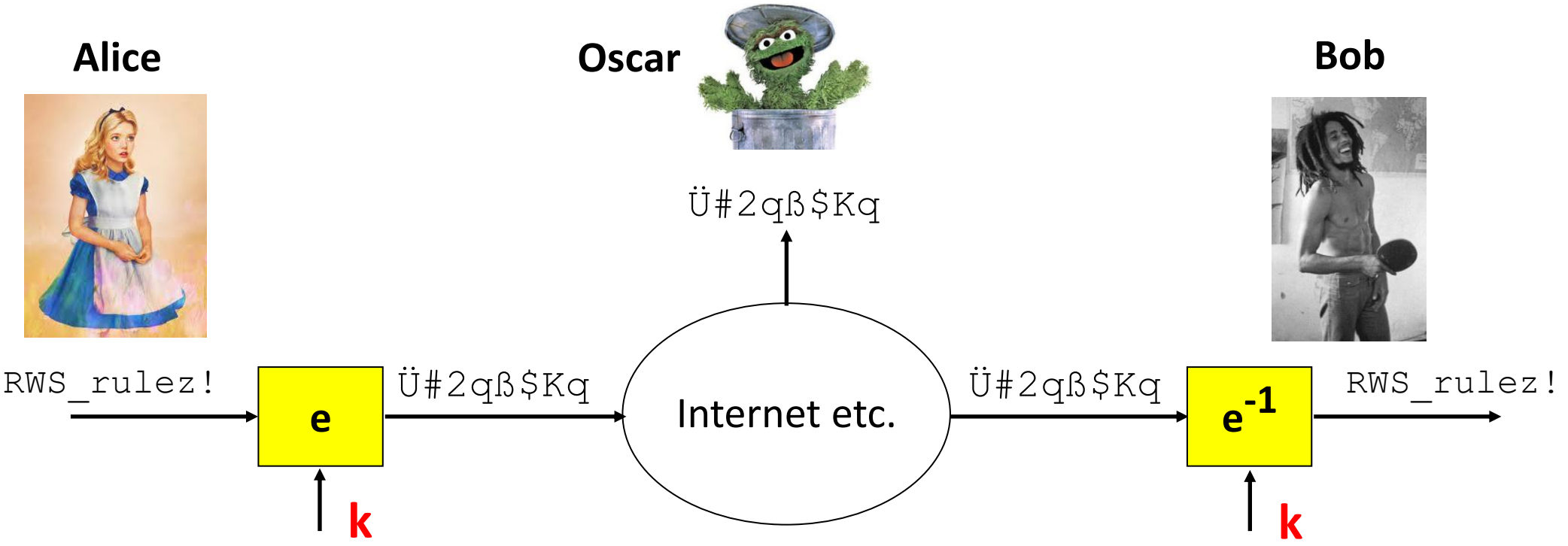
Code:



Goals of this lecture

After this lecture, you will …

- know physical methods to inject faults into processors
- understand techniques to break secure crypto from faulty computation result
- know (some) countermeasures



Classical security assumption

1. Attacker knows x and $e_k(x)$
2. Attacker deduces ***k*** via mathematical, protocol, or brute-force attacks

„Problem“ for adversary

Today‘s crypto algorithms and protocols are often secure against classical attacks:

AES, XSalsa20, RSA, ECC, …

Embedded reality:

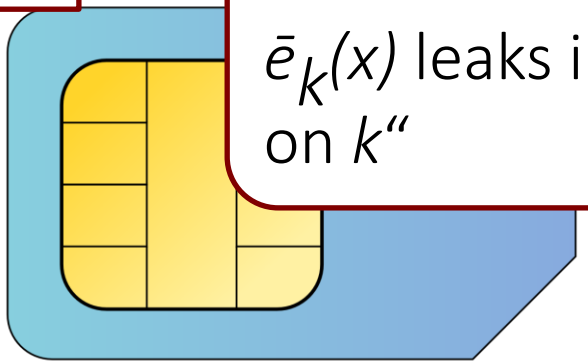
Adversa
controls

2. Control: fault injection

- Power/clock glitch
- EM pulse
- (Laser) light
- …

Side channel

Fault injection



Intuition: "Faulty output $\bar{e}_k(x)$ leaks information on k"

Intuition: "Power consumption of $e_k(x)$ depends on k"

1. Observe: side channel

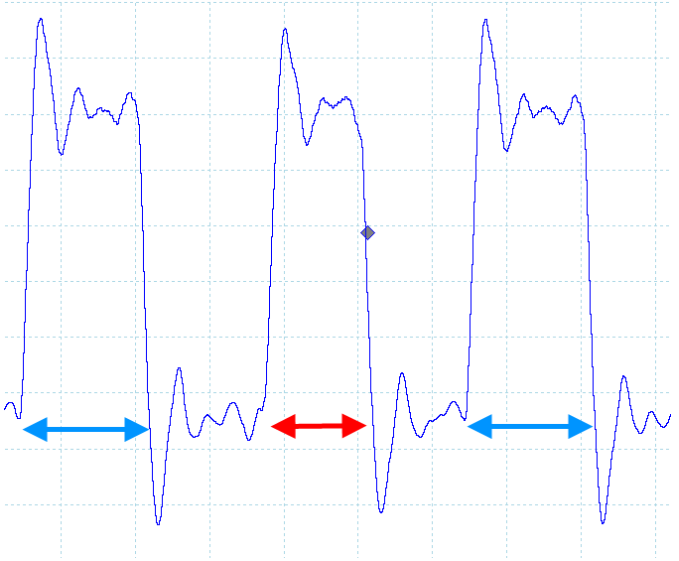
- Power consumption
- EM emanation
- (Cache) timing
- …

Simcard by: Dake@fr.wikipedia.org

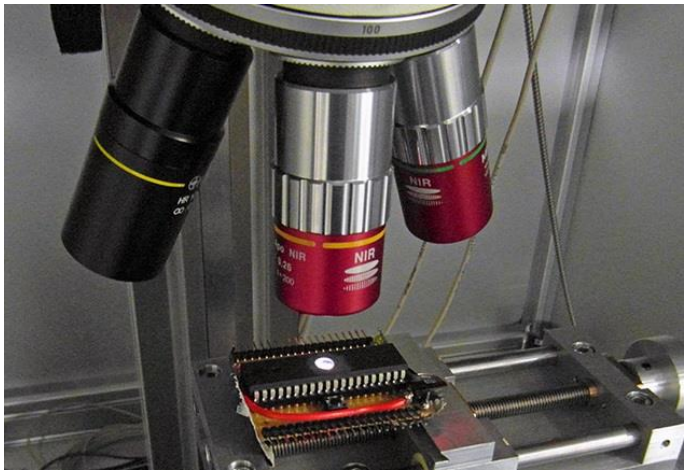
Injecting faults (examples)

power

clock



laser

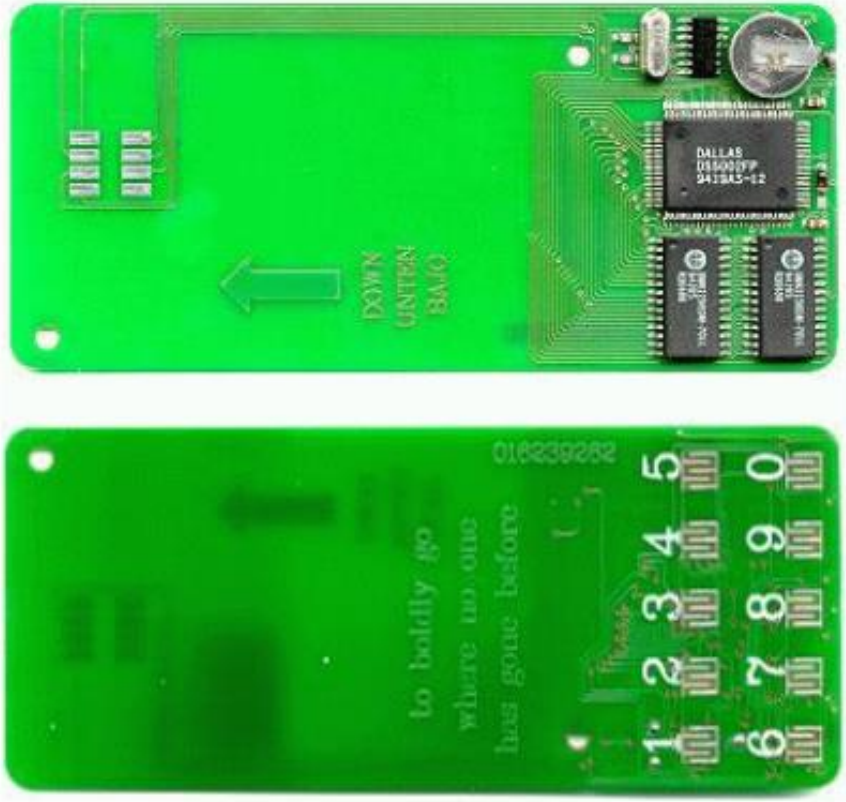


EM

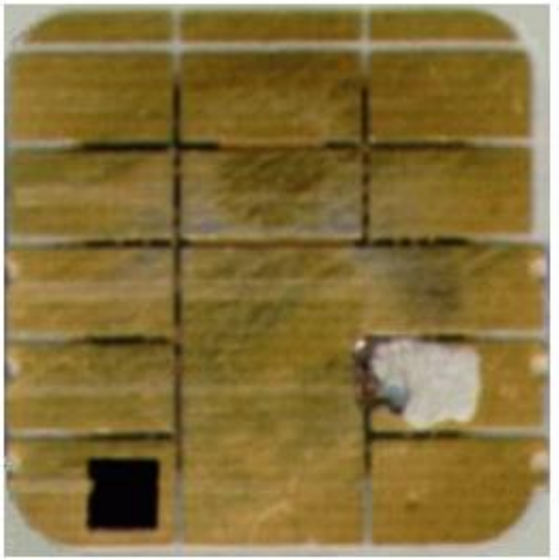
https://github.com/newaetech/chipshouter-picoemp

https://www.cl.cam.ac.uk/~mgk25/vc-slides.pdf

Some Pay-TV Pirate Devices

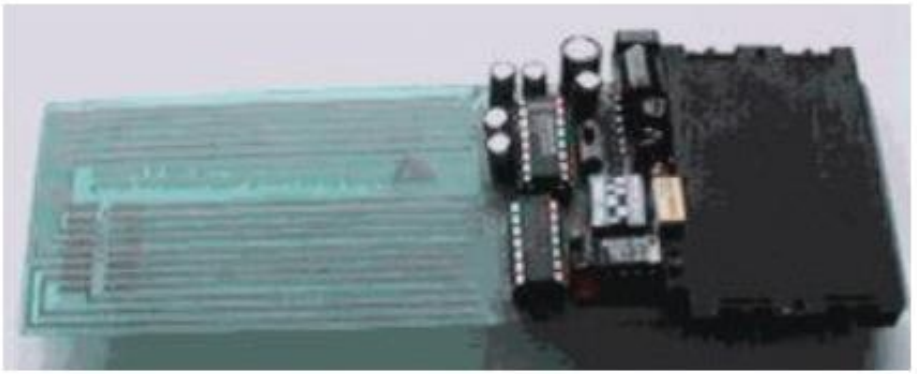


"Battery-powered smartcard", Megasat Bochum



Conductive silver ink attack on BSkyB P10 card (top), with card CPU replaced by external DS5002FP (right)

BSkyB P9 deactivation blocker



ISO 7816 to RS-232 adapter (Season7)

https://www.aliexpress.com/item/1005006607335880.html

https://abrites.com/products/abrites-rh850-v850-programmer

ZN085 - Abrites Programmer RH850/V850

SKU: ZN085

Category Additional Cables And Adapters

€550.00 (Excl. VAT)

The revolutionary ABRITES tool for reading data from locked RH850/V850 processors.

An AVDI with active AMS is required to complete all procedures. ×

LOGIN TO SHOP

Fault models

- In a nutshell this means: how can the attacker affect/change internal values?
- **Spatial location**: specific bit/byte/word/instruction vs. random position/width
- **Temporal position**: Number of affected cycles / instructions, timing jitter, …
- **Effect**:
  - bit(s) flip (1 → 0, 0 → 1)
  - bit(s) stuck at zero/one (e.g. 0 → 0, 1 → 0)
  - statistical (e.g. with 75% chance: 0 → 1, 1 → 1)
  - skip instructions
- …

Bypassing PIN check with FI

Fault model

Location: specific instruction

Effect: Skip instruction

Temporal: specific instruction

```
int c = memcmp(entered_pin, stored_pin, pin_len);

if(c != 0)
{
    return -1;
}

// Code continues ...
```

Skip return / if / …

Live-Demo

"Anything that can go wrong,
will go wrong"

EM fault injection

https://github.com/martinjthompson/glitch-target

Example code

```
// ...
while(application_good)
{
      // ...
      application_good = check_application();
      // ...
}
// ...
// Security bypassed
pwm_chirp(slice, channel, 1600, 400);
```

A generic attack on keyed algorithms

- One of the earliest published fault attacks – works generically, but has specific assumptions
- **Given**: any encryption function $c = e_k(p)$
- **Assumptions**:
  - Attacker can repeatedly call function for constant p
  - Attacker can precisely target any bit of k with a stuck-at-zero fault ($0 \rightarrow 0$, $1 \rightarrow 0$)

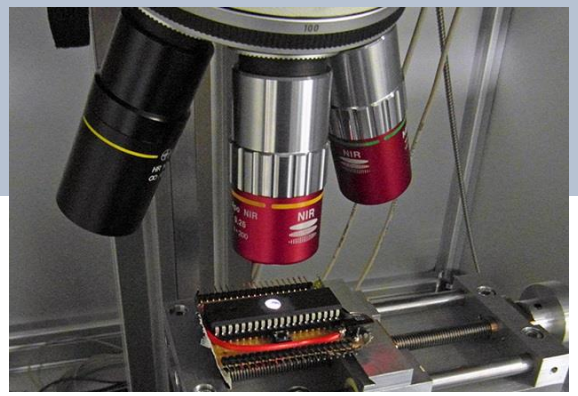
The generic attack

Fault model
Location: specific bit of k
Effect: bit stuck-at-zero
Temporal: during key load

1. Get fault-free ciphertext $c^{ref} = e_k(p)$
2. Now, inject the fault for bit 0 of k: $c^0 = e^0{}_k(p)$
   - If $c^0 == c^{ref}$: fault had no effect, key bit 0 is 0
   - Else: fault flipped a one to zero, key bit 0 is 1
3. Repeat for bit 1, 2, 3, …

Complexity: we need as many faults as key bits plus negligible computation

The generic attack - remarks

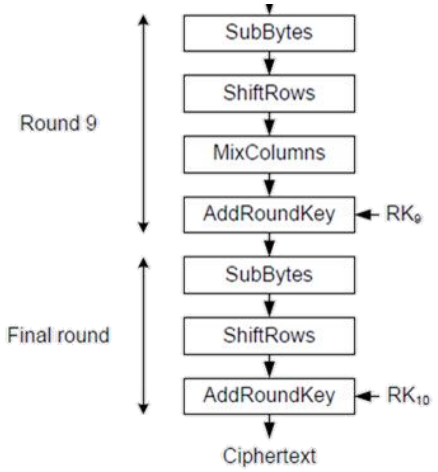


- Simple attack, but effective and practical
- Works in the same way for stuck-at-one faults
- Idea can be extended in various ways:
  - Can also work if you can force larger values to all-zero / all-one: set most of key to zero, brute-force the rest
  - For some smartcards, you don't need fault injection: key can be programmed byte-wise; attack can be adapted to that scenario

$$q = \gcd\left(\overline{s}^{e} - x,\, n\right),\, p = \frac{n}{q}$$

$$q = \gcd\left(s - \overline{s},\, n\right),\, p = \frac{n}{q}$$

Fault injection tailored to crypto algorithms



The CRT-RSA algorithm

To compute an RSA signature $s = x^d \bmod n$

The following quantities are pre-computed once:

$d_p \leftarrow d \mod p-1$

$d_q \leftarrow d \mod q-1$

$c_p \leftarrow q^{-1} \mod p$

$c_q \leftarrow p^{-1} \mod q$

Then, compute:

$x_p \leftarrow x \mod p$

$x_q \leftarrow x \mod q$

$\overline{s_p} \leftarrow x_p^{d_p} \mod p$

$s_q \leftarrow x_q^{d_q} \mod q$

Fault model

Location: random

Effect: any

Temporal: only one of the subexponentiations

Recombine result:

$\overline{s} \leftarrow [q \cdot c_p] \cdot \overline{s_p} + [p \cdot c_q] \cdot s_q \mod n$

Bellcore and Lenstra attack

- Bellcore (initial attack):

$$q = \gcd\left(s - \overline{s},\, n\right),\; p = \frac{n}{q}$$

Correct sig of x

Faulty sig of x

- Lenstra (from a “letter”):

$$q = \gcd\left(\overline{s}^{e} - x,\, n\right),\; p = \frac{n}{q}$$

Faulty sig of x

Known

Whiteboard

"Anything that can go wrong, will go wrong"

Deriving the Bellcore attack

Examples in the real world

- Weimer observed in 2015 that such random faults sometimes happen accidentally
- Scanned Internet and compromised TLS keys:

Vendor	Keys	Geo	PKI	Rate
Citrix	2	DE	yes	medium
Hillstone Networks	231	CN	no	low
	1	CZ	no	low
	1	PL	no	low
	1	TH	no	low
	1	US	no	low
Alteon/ Nortel	1	US	expired	high
	1	US	no	high
Viprinet	1	NL	no	always
QNO	2	CN	no	medium
	1	TW	no	medium
ZyXEL	4	AT	no	low
	1	CH	no	low
	1	DE	no	low
	2	DK	no	low
	1	FR	no	low
	7	IE	no	low
	1	IT	no	low
	2	NL	no	low
	1	SE	no	low
	5	UK	no	low
	1	US	no	low
BEJY	1	DE	yes	low (?)
Fortinet	1	US	no	very low
	1	IN	no	very low

Implementation	Verification
cryptlib 3.4.2	disabled by default
GnuPG 1.4.18	yes
GNUTLS	see libgcrypt and Nettle
Go 1.4.1	no
libgcrypt 1.6.2	no
Nettle 3.0.0	no
NSS	yes
ocaml-nocrypto 0.5.1	no
OpenJDK 8	yes (see text)
OpenSSL 1.0.1l	yes (see section 1.3.1)
OpenSwan 2.6.44	no
PolarSSL 1.3.9	no

Table 1: RSA-CRT and signature verification

https://people.redhat.com/~fweimer/rsa-crt-leaks.pdf

Fault attacks on AES

Attacker: Given

$y = \mathrm{AES}_k(x)$ and $y' = \mathrm{AES}_k(x)$

compute ***k***

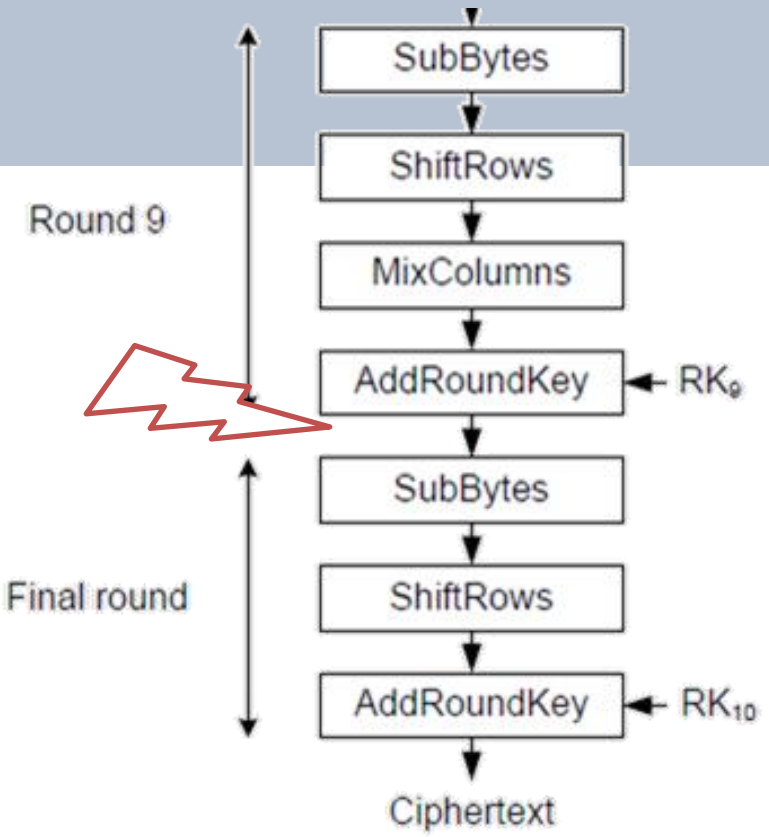
Attack depends heavily on

1. Crypto algorithm
2. Fault model
   - Random
   - Single bit (unknown)
   - Single bit (known)
   - …

Fault attacks on AES

Attacker model:

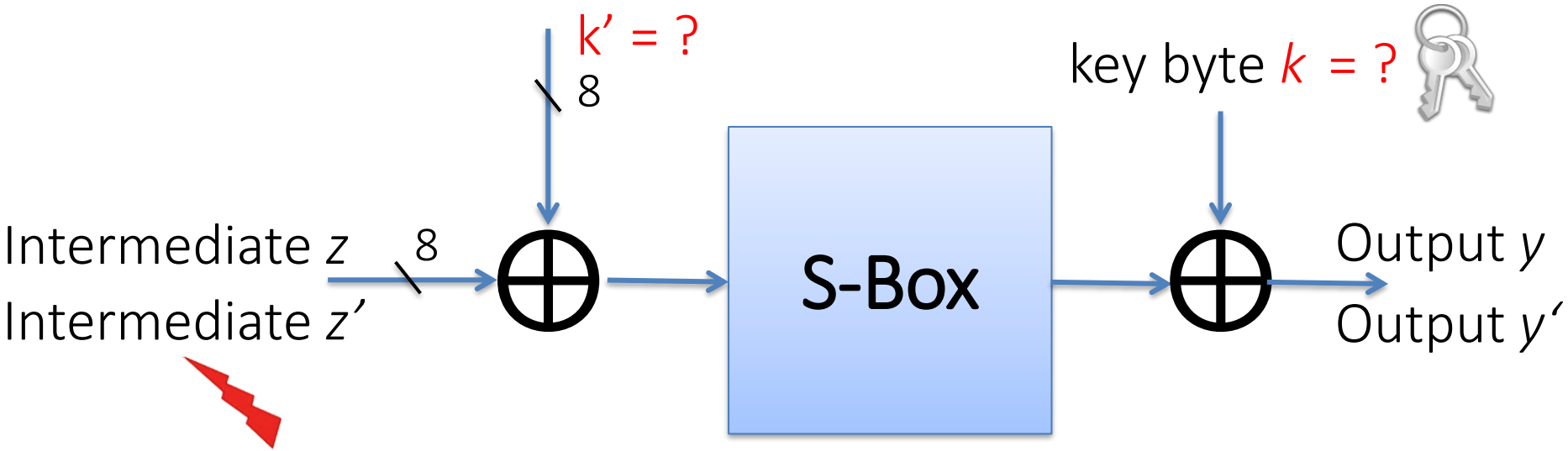
- Encrypt the same plaintext multiple times
- Toggle a bit in last round at known location (powerful model!)



Fault model

Location: specific bit
Effect: flip bit
Temporal: before SubBytes in final round

Differential fault analysis (DFA) for AES

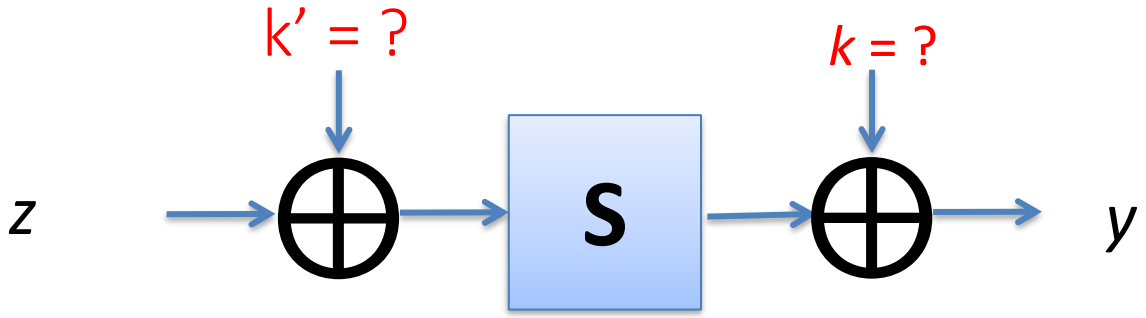


Attacker gets *y, y'* but does NOT know *z, z'* - but the difference:

$z \oplus z' = \Delta$ = mask for flipped bit (e.g. 0b10000000 = 0x80)

Allows differential, bytewise attack!

Differential fault analysis (DFA) for AES



Let's write equations:

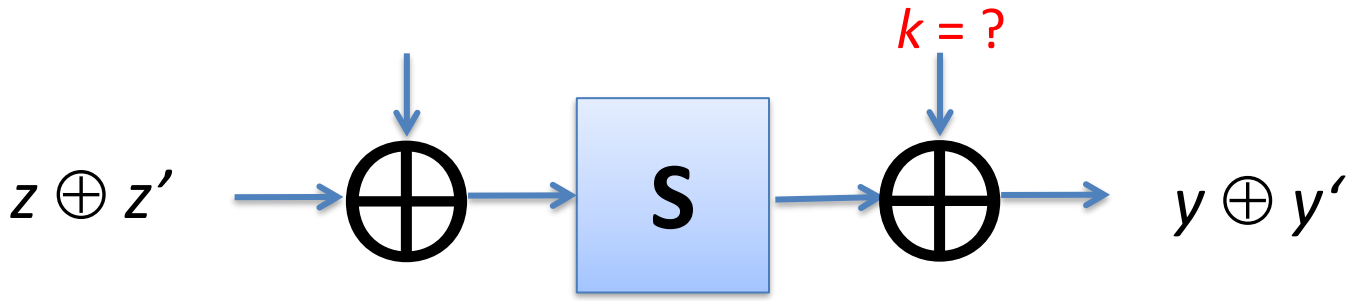
$y = S(k' \oplus z) \oplus k \rightarrow z = S^{-1}(k \oplus y) \oplus k'$

$y' = S(k' \oplus z') \oplus k \rightarrow z' = S^{-1}(k \oplus y') \oplus k'$

Hence:

$\Delta = z \oplus z' = S^{-1}(k \oplus y) \oplus S^{-1}(k \oplus y')$

Differential fault analysis (DFA) for AES



- Injected fault: $\Delta = 0x80$ (MSBit fault)
- Observed outputs: $y_1 = 0xFA$ and $y_1' = 0x14$

Now, find all key candidates k that satisfy

$$S^{-1}(k \oplus y_1) \oplus S^{-1}(k \oplus y_1') = \Delta = 0x80$$

→ only 2 key candidates: $k = 0x4E$ and $k = 0xA0$

Another pair: $y_2 = 0x37$ *and* $y_2' = 0x5B$

→ only 1 remaining $k = 0x4E$

The state of DFA on AES

- DFA on AES has improved over the years
- Best attack: https://eprint.iacr.org/2009/575
- Implementation: https://github.com/Daeinar/dfa-aes
- Recover full key from one correct and one faulty ciphertext (same plaintext) in minutes

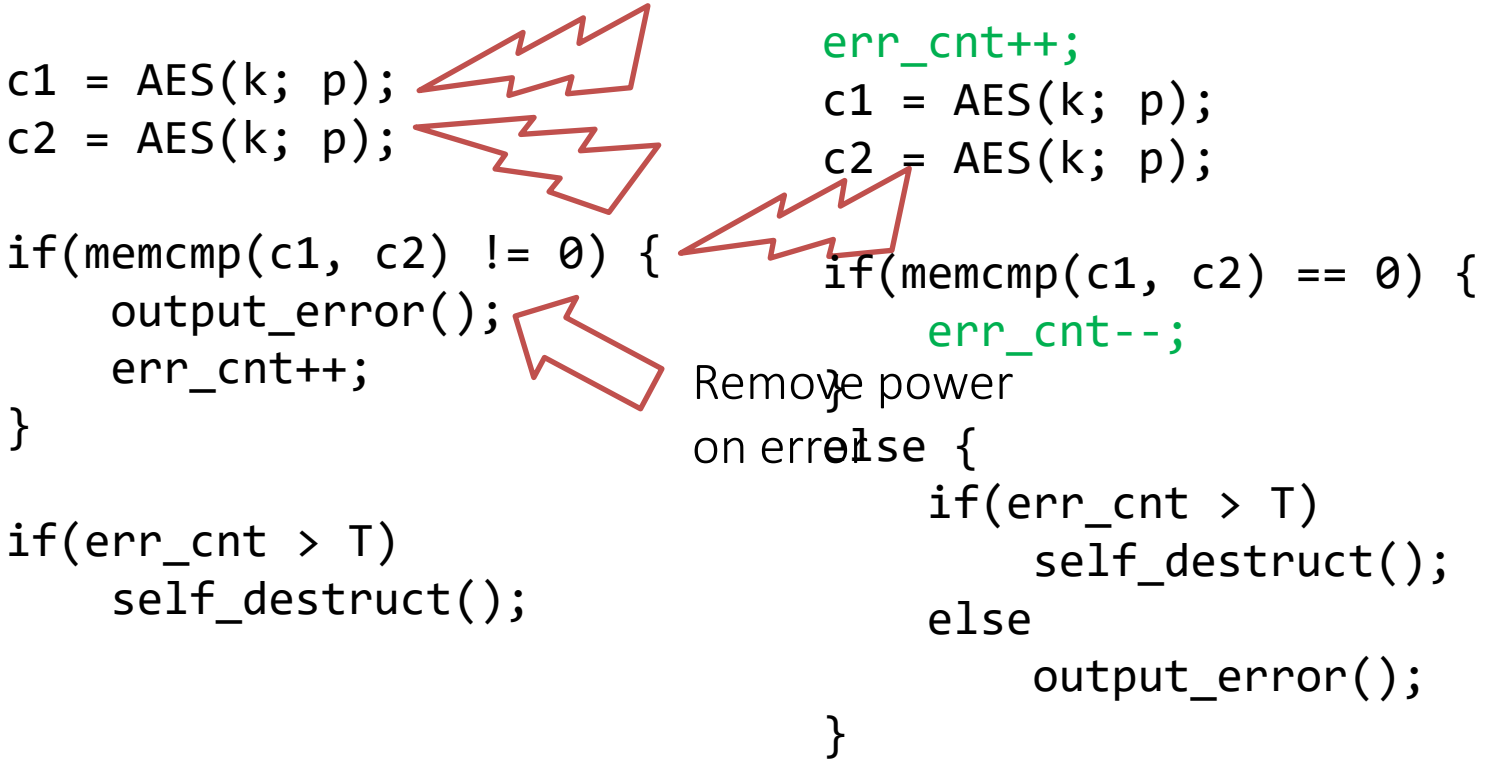
Fault model
Location: any single byte
Effect: random
Temporal: at the input to the 8^{th} round

(Some) countermeasures

- Detection-based:
  - Detect injection of fault (monitor environmental conditions)
  - Detect faulty result: compute twice (or more) and compare. Either in parallel or sequential.
  - Detected “unusual usage patterns” (e.g. error counter)
- Algorithmic:
  - RSA / ECC: Verify signature after signing
  - AES: Backward rounds
  - Randomization in time (harder to inject fault)

Countermeasures: A cautionary note

- Most countermeasures can be bypassed when applied in isolation
- Countermeasure impl. can also have weaknesses:



Take-home points

- Fault injection is independent of mathematical security (or of your code being bug-free)
- If there's a cipher, there is usually an efficient fault attack on it
- There are various ways to inject faults with low-cost tools
- Countermeasures are hard/costly
- In the next lecture, we'll see more examples

Thanks!

Questions / discussion?

d.f.oswald@bham.ac.uk